SEG3502 – Lab 6

Messagerie & Services Web REST

Les objectifs de ce laboratoire sont (1) une introduction à la messagerie avec le avec le Broker de messages RabbitMQ et (2) une introduction à la programmation des services Web RESTful avec Angular et Springboot. Dans la première partie, nous créons une application de chat simple à l'aide de RabbitMQ. Nous développons une application Angular pour consommer un service Web disponible publiquement et créons une API de service Web RESTful à l'aide de Springboot, dans la deuxième partie.

1. Application Chat avec RabbitMQ

RabbitMQ est un broker de messages open source qui supporte AMQP (Advanced Message Queuing Protocol). Nous utilisons RabbitMQ pour stocker et relayer les messages entre correspondants dans une application de clavardage (chat).

Le code source de l'application est disponible à https://github.com/stephanesome/rabbit-chat.git.

Setup de Docker

Nous utilisons un déploiement Docker de RabbitMQ. Docker est une plateforme de conteneurisation. Il permet de créer des conteneurs qui regroupent des applications avec toutes les dépendances nécessaires, telles que des bibliothèques. Nous pouvons également utiliser Docker pour installer des images pré-packagées pour des composants tels que les serveurs Web, les bases de données ou le middleware orienté message utilisé dans ce laboratoire. Cela simplifie la configuration et la gestion de ces composants.

Installez Docker Community Edition (Docker CE) pour votre plate-forme en suivant les instructions à l'adresse: https://docs.docker.com/get-docker/.

RabbitMQ

Exécutez l'image Docker de RabbitMQ avec la commande

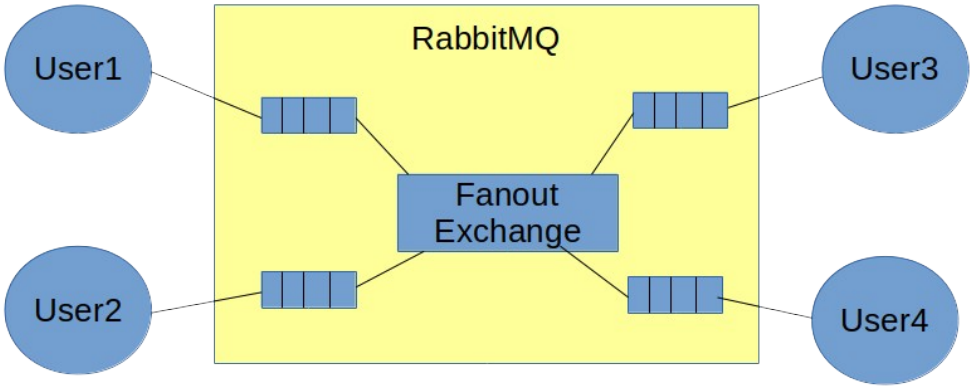
```
docker run -d --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```

La première exécution prendra du temps car docker téléchargera et configurera l'image.

Simple Chat

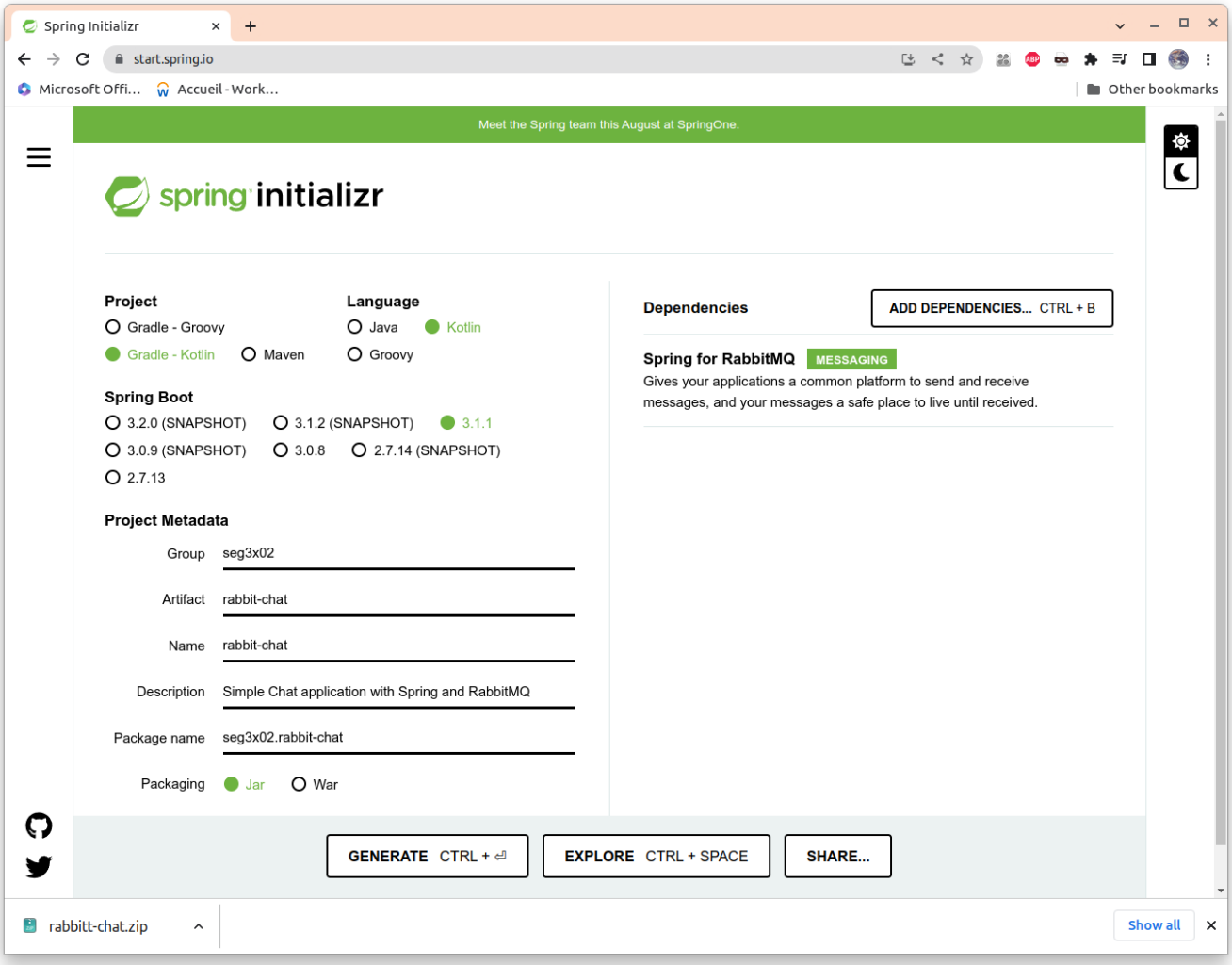
Nous allons créer un programme Simple Chat pour démontrer le développement d'applications Spring Boot et l'utilisation de la messagerie.

Notre Chat Simple utilise RabbitMQ pour stocker et relayer les messages entre plusieurs participants du chat. Nous utiliserons un Exchange Fanout qui copie le même message dans toutes ses files d'attente liées. Chaque participant au chat aura sa propre file d'attente connectée à l'échange.



Générer l'application

Nous utilisons Spring Intializr (https://start.spring.io/) pour la génération de l'application.



- Sélectionnez Gradle Projet comme type de projet et Kotlin comme langage
- Laissez la version Spring Boot sélectionnée par défaut
- Entrez les métadonnées (Metadata) du projet
- Dans la section Dependencies, cliquez sur Add. Sélectionnez Spring for RabbitMQ.
- Générez le projet et téléchargez le fichier compressé zip généré.

Configuration RabbitMQ

Créez un fichier source Kotlin dans le package seg3x02.rabbitchat appelé RabbitMQConfiguration. Modifiez comme suit.

```
1.  package seg3x02.rabbitchat
2.
3.  import org.springframework.amqp.core.*
4.  import org.springframework.context.annotation.Bean
5.  import org.springframework.context.annotation.Configuration
6.
7.  @Configuration
8.  class RabbitMQConfiguration {
9.      @Bean
10.     fun queue(): AnonymousQueue {
11.         return AnonymousQueue()
12.     }
13.
14.     @Bean
15.     fun exchange(): FanoutExchange {
16.         return FanoutExchange("chatfan")
17.     }
18.
19.     @Bean
20.     fun binding(exchange: FanoutExchange,
21.                 queue: Queue): Binding {
22.         return BindingBuilder.bind(queue)
23.                 .to(exchange)
24.     }
25. }
```

Il s'agit d'un fichier de configuration qui définit les Beans (composants injectables Spring). Nous créons une file d'attente (Queue - lignes 9 à 12), un Exchange (lignes 14 à 17) et une liaison de la file d'attente et de l'échange (Binding - lignes 19 à 24). Un AnonymousQueue une spécialisation de classe Queue est utilisé ici. RabbitMQ attribuera un nom aléatoire à la file d'attente et s'assurera qu'elle est supprimée lorsque le serveur de messages est mis hors tension.

Message Receiver

Un Listener est enregistré pour une file d'attente afin de recevoir des messages de celle-ci. Créez une classe Kotlin MessageReception dans le package seg3x02.rabbitchat et modifiez-la comme suit.

```
1.  package seg3x02.rabbitchat
2.
3.  import org.springframework.amqp.rabbit.annotation.RabbitListener
4.  import org.springframework.beans.factory.annotation.Value
5.  import org.springframework.stereotype.Component
6.
7.  @Component
8.  class MessageReception {
9.      @Value("\${chat.alias}")
10.     val USER: String=""
11.
12.     @RabbitListener(queues = ["#{queue.name}"])
13.     fun receive(msg: String) {
14.         val user = msg.split(":")[0]
15.         if (!this.USER.equals(user)) println(msg)
16.     }
17. }
```

Une fonction d'écoute de message est annotée avec @RabbitListener. L'annotation spécifie les noms des files d'attente écoutées. Ici, nous nous référons au nom de la file d'attente créée dans RabbitMQConfiguration et enregistrée en tant que Bean. Tous les messages de la file d'attente seront passés à la fonction de receive (lignes 12 à 16) en tant que paramètre msg.

Nous voulons que nos utilisateurs de chat puissent s'identifier dans leurs messages. À cet effet, nous ajoutons une propriété d'application appelée chat.alias qui capture un alias de l'utilisateur. Cet alias sera transmis sur la ligne de commande lors du démarrage de l'application. La propriété de l'application est injectée dans la variable USER à la ligne 10. La fonction de réception vérifie qu'un message reçu n'est pas du récepteur avant de l'afficher (lignes 14-15).

Modifiez le fichier application.properties de src/resources, comme suit

```
1.  spring.main.banner-mode=off
2.  logging.level.root=OFF
3.
4.  chat.alias=ANONYMOUS
```

La propriété d'application chat.alias est définie et affectée une valeur par défaut (ligne 4). Nous désactivons également l'affichage de la bannière Spring et des messages de journalisation lorsque l'application est exécutée.

Producteur de Message

Le producteur de messages reçoit les messages saisis par l'utilisateur et les envoie à Exchange. Créez une classe Kotlin MessageProducer dans le package seg3x02.rabbitchat et modifiez-la comme suit.

```
1.  package seg3x02.rabbitchat
2.
3.  import org.springframework.amqp.core.FanoutExchange
4.  import org.springframework.amqp.rabbit.core.RabbitTemplate
```

```
5.  import org.springframework.beans.factory.annotation.Value
6.  import org.springframework.boot.CommandLineRunner
7.  import org.springframework.stereotype.Component
8.  import kotlin.system.exitProcess
9.
10. @Component
11. class MessageProducer(val template: RabbitTemplate, val exchange: FanoutExchange):
    CommandLineRunner {
12.     @Value("\${chat.alias}")
13.     val USER: String=""
14.
15.     override fun run(vararg args: String?) {
16.         println("**** You are connected to Simple Chat ****")
17.         println("**** Enter #BYE to terminate          ****")
18.         println("*****************************************")
19.         println()
20.         var msg = readLine();
21.         while (msg != null && msg != "#BYE") {
22.             template.convertAndSend("chatfan", "", "$USER: $msg")
23.             msg = readLine()
24.         }
25.         println("Shutting Down ...")
26.         exitProcess(0)
27.     }
28. }
```

Le MessageProducer implémente un CommandLineRunner. Un RabbitTemplate et le FanoutExchange sont injectés dans le constructeur de classe et utilisés dans la fonction run pour envoyer le message de l'utilisateur (ligne 22). L'alias de l'utilisateur défini comme propriété d'application est injecté dans la variable USER (lignes 12-13) et concaténé au message.

Construction et exécution

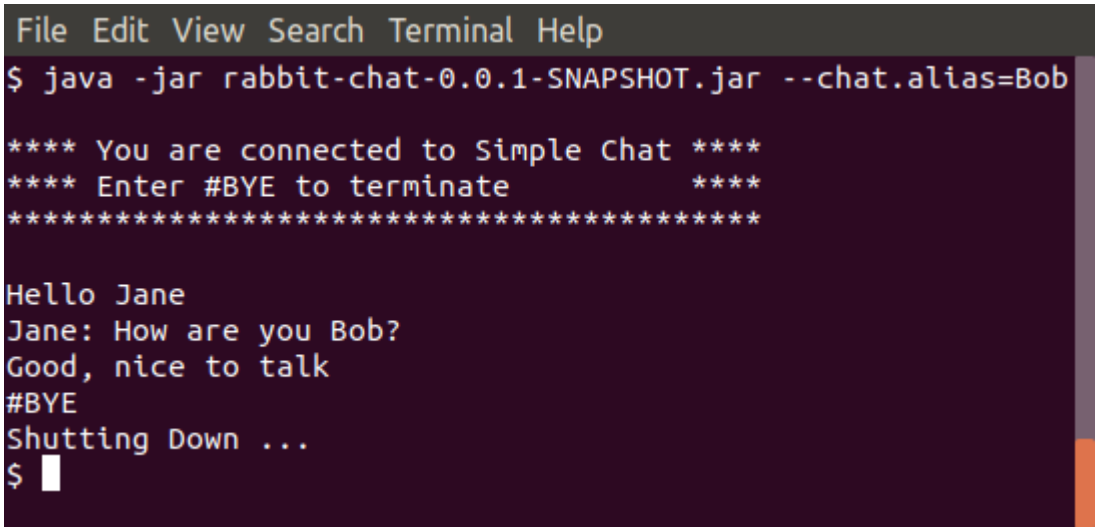
Votre IDE fournit probablement un moyen intégré pour construire et exécuter des applications Springboot. Cependant, nous utiliserons la ligne de commande pour créer un Uber Jar (un Jar avec toutes ses dépendances et exécutable).

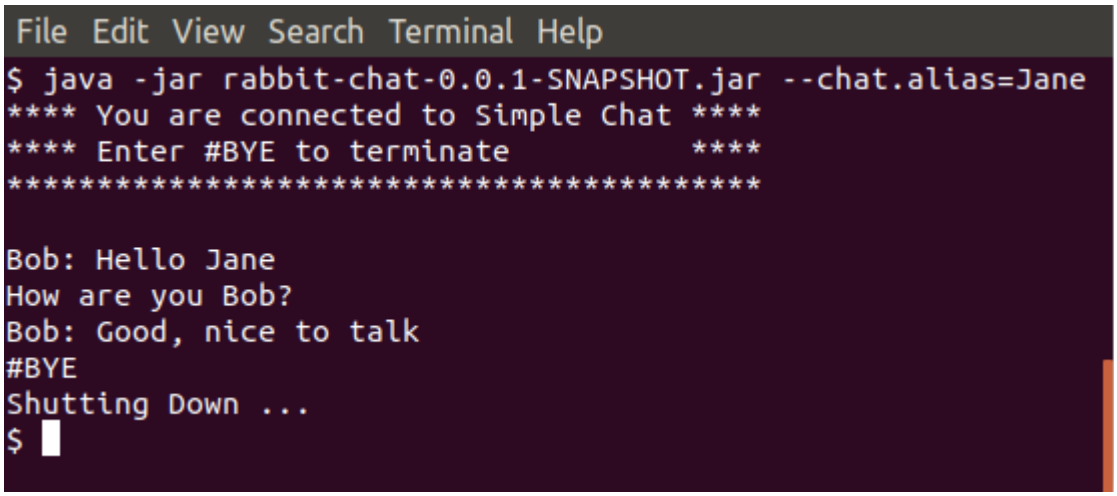
Dans le dossier racine du projet, entrez la commande `./gradlew bootJar` cela construira le projet et générera un fichier Jar dans le sous-dossier build/libs.

Assurez-vous que RabbitMQ est en cours d'exécution et ouvrez quelques terminaux pour simuler plusieurs utilisateurs. Exécutez le Jar créé dans chacun avec la commande

```
java -jar rabbit-chat-0.0.1-SNAPSHOT.jar --chat.alias=Bob
```

Avec `rabbit-chat-0.0.1-SNAPSHOT.jar`, le Jar généré et `Bob` l'alias de l'usager.





2. Weather Application (météo)

Nous allons développer une application Angular pour afficher les conditions météorologiques actuelles d'une ville. L'application présente un formulaire dans lequel l'utilisateur peut spécifier la ville d'intérêt et éventuellement sélectionner un pays où se trouve cette ville. Lors de la soumission, l'application affiche les conditions météorologiques actuelles de cette ville.

Nous utiliserons une API de service Web RESTful fournie par Open Weather Map (`http://openweathermap.org/`) pour obtenir les conditions météorologiques courantes.

Le code source de cette application est disponible à https://github.com/stephanesome/weatherApp.git.

Clé d'API

Vous avez besoin d'une clé API d'OpenWeatherMap que vous pouvez obtenir en vous abonnant à un compte gratuit. Accédez à `http://openweathermap.org/api`, sélectionnez le bouton

Subscribe de *Current Weather Data*. Faites défiler jusqu'à la section *Free Access* et cliquez sur *Get API Key* pour *Current weather and forecast*.

Configuration du projet

Générez une application angular (`ng new weather-application`). Nous utiliserons CSS comme format de feuille de style.

Ajoutez bootstrap au projet (`ng add @ng-bootstrap/ng-bootstrap`).

Nous utiliserons également un composant de Angular Material (`https://material.angular.io/`) pour sélectionner les pays.

- Ajoutez Angular Material avec la commande `ng add @angular/material`. Sélectionnez les choix par défaut.
- Ajoutez l'extension ng-country-select au projet (https://github.com/wlucha/ng-country-select) `ng add @wlucha/ng-country-select`
- Importez HttpClient dans `src/app/app.config.ts`

```
1.  import {
2.    ApplicationConfig,
3.    importProvidersFrom,
4.    provideBrowserGlobalErrorListeners,
5.    provideZoneChangeDetection
6.  } from '@angular/core';
7.  import { provideRouter } from '@angular/router';
8.
9.  import { routes } from './app.routes';
10. import {provideHttpClient} from '@angular/common/http';
11.
12. export const appConfig: ApplicationConfig = {
13.   providers: [
14.     provideBrowserGlobalErrorListeners(),
15.     provideZoneChangeDetection({ eventCoalescing: true }),
16.     provideRouter(routes),
17.     provideHttpClient()
18.   ]
19. };
```

Modèle Weather

Nous créons une classe de modèle pour représenter les informations météorologiques retournées par l'appel d'API. Exécutez la commande `ng generate class model/weather --type=model` pour générer une classe de modèle. Modifiez `src/app/service/open-weather.service.ts` comme suit

```
1.  export class Weather {
2.    mainCondition: string;
```

```
    temperature: number;
    pressure: number;
    humidity: number;
    windspeed: number;
    city: string;
    country: string;

    constructor(obj?: any) {
      this.mainCondition = obj && obj.mainCondition || null;
      this.temperature = obj && obj.temperature || null;
      this.pressure = obj && obj.pressure || null;
      this.humidity = obj && obj.humidity || null;
      this.windspeed = obj && obj.windspeed || null;
      this.city = obj && obj.city || null;
      this.country = obj && obj.country || null;
    }
}
```

La classe Weather définit des champs pour les informations météo qui nous intéressent. Le constructeur crée une instance à partir des informations disponibles.

Service d'accès à l'API Weather

Nous utilisons un service pour interagir avec l'API OpenWeather. Cela permet au reste du code d'être extrait des détails de l'interaction. Cela pourrait également faciliter la substitution d'un autre service pour les informations météo.

Générez un service `ng generate service service/open-weather-service`.
Éditez `src/app/service/open-weather-service.ts` comme suit.

```
import { Injectable } from '@angular/core';
import {HttpClient, HttpErrorResponse, HttpParams} from '@angular/common/http';
import {Observable, throwError} from 'rxjs';
import {catchError} from 'rxjs/operators';

const baseUrl = 'http://api.openweathermap.org/data/2.5/';
const APPID_HEADER = 'YOUR_API_ID';
const resource = 'weather';

@Injectable({
  providedIn: 'root'
})
export class OpenWeatherService {
  private httpClient: HttpClient = inject(HttpClient);

  public getWeatherAtCity(city: string, country: string): Observable<unknown> {
```

```
17.     const params = new HttpParams().set('q', city + ',' + country).set('appid', APPID_HEADER);
18.     const options = {params, responseType: 'json' as const};
19.     return this.httpClient.get(baseUrl + resource, options).pipe(
20.       catchError(this.handleError)
21.     );
22.   }
23.
24.   private handleError(error: HttpErrorResponse): Observable<never> {
25.     return throwError(() =>
26.       'Error - Unable to retrieve Weather Condition for Specified City.');
27.   }
28. }
```

Remplacez 'YOUR_API_ID' par l'ID d'API que vous avez obtenu de openweathermap.org.

Le service utilise HTTPClient (injecté à la ligne 14) pour émettre des requêtes vers l'API. Assurez-vous de spécifier votre Token API Open Weather à la ligne 7. La fonction getWeatherAtCity construit la requête en définissant les paramètres (ligne 17) avec la ville et le pays ainsi que et le jeton API, et en définissant le type de réponse attendu (ligne 18). La requête est envoyée en tant que méthode HTTP GET.

Les requêtes HTTP sont traitées de manière asynchrone. Un appel HTTPClient retourne un Observable auquel le requérant s'abonne. Nous utilisons l'opérateur pipe pour transmettre toute erreur retournée à la fonction handleError (lignes 24-27). La fonction assure simplement que nous avons un traitement d'erreur standard en retournant un Observable qui émet une notification d'erreur.

Composant App

Le composant App est le seul composant utilisé dans cette application. Il présente un formulaire pour obtenir la ville et le pays d'intérêt d'un utilisateur, utilise le service Weather pour obtenir des données météorologiques et les affiche à l'utilisateur.

Éditez la classe de composant App (`src/app/app.ts`) comme suit.

```
1.  import {Component, inject, OnInit} from '@angular/core';
2.  import {Weather} from './model/weather.model';
3.  import {
4.    FormBuilder,
5.    FormControl,
6.    FormGroup,
7.    ReactiveFormsModule
8.  } from '@angular/forms';
9.  import {OpenWeatherService} from './service/open-weather-service';
10. import {Country, CountrySelectComponent} from '@wlucha/ng-country-select';
11. import {DatePipe, DecimalPipe} from '@angular/common';
12.
```

```
13. function parseResponse(response: any): Weather {
14.   return new Weather({mainCondition: response.weather[0].main,
15.       temperature: response.main.temp - 273.15,
16.       pressure: response.main.pressure,
17.       humidity: response.main.humidity,
18.       windspeed: response.wind.speed,
19.       city: response.name,
20.       country: response.sys.country
21.     }
22.   );
23. }
24.
25. @Component({
26.   selector: 'app-root',
27.   imports: [ReactiveFormsModule, DecimalPipe, CountrySelectComponent, DatePipe],
28.   templateUrl: './app.html',
29.   styleUrl: './app.css'
30. })
31. export class App  implements OnInit {
32.   protected title = 'weather-app';
33.   weatherForm!: FormGroup;
34.   condition: Weather | null = null;
35.   message: string | null = null;
36.   currentDate!: number;
37.   private formBuilder: FormBuilder = inject(FormBuilder);
38.   private weatherService: OpenWeatherService = inject(OpenWeatherService);
39.   // @ts-ignore
40.   protected country: (FormControl<Country | null> & FormControl);
41.
42.   ngOnInit(): void {
43.     this.weatherForm = this.formBuilder.group({
44.       country: [],
45.       city: []
46.     });
47.     this.country = this.weatherForm.get('country') as FormControl;
48.   }
49.
50.   submit(): void {
51.     const selectedCountry: Country = this.weatherForm.get('country')!.value;
52.     const selectedCity: string = this.weatherForm.get('city')!.value;
53.     this.weatherService.getWeatherAtCity(selectedCity, selectedCountry.alpha2).subscribe({
54.       next: (response: any) => {
55.         this.message = null;
56.         this.currentDate = Date.now();
57.         this.condition = parseResponse(response);
58.       },
59.       error: (error: any) => {this.condition = null; this.message = error; }
60.     });
61.   }
```

```
62. }
```

La fonction parseResponse (lignes 13 à 23) prend un objet JSON, extrait les informations pertinentes et crée un objet de modèle Weather à partir de ces informations. La classe définit un ReactiveForm à utiliser par le modèle HTML et gère la soumission du formulaire. Les réponses de l'observable du service Weather (injecté à la ligne 38) sont analysées et mises à la disposition du modèle HTML (ligne 57) tandis que les erreurs retournées définissent un message d'erreur à afficher (ligne 59).

Éditez le modèle HTML du composant App comme suit.

```
1.  <div>
2.    <h1>Weather Application</h1>
3.  </div>
4.  <div class="container-sm" >
5.    <form [formGroup]="weatherForm" (ngSubmit)="submit()" >
6.      <div class="row" >
7.        <div class="col-sm">
8.          <ng-country-select
9.            [lang]="'en'"
10.           [appearance]="'outline'"
11.           [formControl]="country">
12.         </ng-country-select>
13.       </div>
14.       <div class="col-sm">
15.         <input type="text" class="form-control" name="city" placeholder="City" formControlName="city">
16.       </div>
17.       <div class="col-sm">
18.         <button type="submit" class="btn btn-success">Check Weather</button>
19.       </div>
20.     </div>
21.   </form>
22. </div>
23. @if(message) {
24.   <div class="container">
25.     <div id="message">{{message}}</div>
26.   </div>
27. }
28. @if(condition) {
29.   <div class="container">
30.     <div id="city">Current Conditions in {{condition.city}} {{condition.country}} - {{currentDate | date:
    'medium' }}</div>
31.     <br>
32.     <div class="container-sm">
33.       <div class="row">
34.         <div class="col-sm">
35.           <p>Main Condition:</p>
36.         </div>
37.         <div class="col-sm">
```

```
38.           <p>{{condition.mainCondition}}</p>
39.         </div>
40.         <div class="col-sm">
41.           <p>Temperature:</p>
42.         </div>
43.         <div class="col-sm">
44.           <p>{{condition.temperature | number}} Celsius</p>
45.         </div>
46.       </div>
47.       <div class="row">
48.         <div class="col-sm">
49.           <p>Pressure:</p>
50.         </div>
51.         <div class="col-sm">
52.           <p>{{condition.pressure | number}} hPa</p>
53.         </div>
54.         <div class="col-sm">
55.           <p>Humidity:</p>
56.         </div>
57.         <div class="col-sm">
58.           <p>{{condition.humidity | number}} %</p>
59.         </div>
60.       </div>
61.       <div class="row">
62.         <div class="col-sm">
63.           <p>Wind Speed:</p>
64.         </div>
65.         <div class="col-sm">
66.           <p>{{condition.windspeed | number}} meter/sec</p>
67.         </div>
68.         <div class="col-sm">
69.           <p><br></p>
70.         </div>
71.         <div class="col-sm">
72.           <p><br></p>
73.         </div>
74.       </div>
75.     </div>
76.   </div>
77. }
```

Nous pouvons voir la composante de sélection de pays (ng-country-select) utilisée aux lignes 8-12.

Éditez le fichier de style CSS du composant App comme suit.

```
1.  #city {
2.    -webkit-box-shadow: 5px 5px 15px 5px #000000;
3.    box-shadow: 5px 5px 15px 5px #000000;
4.    background-color: darkgrey;
5.    color: blue;
```

```
    font-weight: bold;
}

#message {
  -webkit-box-shadow: 5px 5px 15px 5px #000000;
  box-shadow: 5px 5px 15px 5px #000000;
  background-color: beige ;
  color: red;
  font-weight: bold;
  font-size: large;
}
```

3. Service de convertisseur de température (Temperature Converter)

Nous implémentons un service Web RESTful pour la conversion de température avec Springboot. Le service expose les opérations suivantes:

- ***GET temperature-converter/celsius-fahrenheit/{celsius}*** pour retourner la valeur en Fahrenheit de la valeur *celsius*, et
- ***GET temperature-converter/fahrenheit-celsius/{fahrenheit}*** pour retourner la valeur en Celsius de la valeur *fahrenheit*.

Nous présentons dans cette section une implémentation de l'API REST basée sur le framework Spring MVC. Ce framework est construit sur la technologie Servlet API, le framework original pour le Web en Java. Il a été inclus dans Spring Framework depuis le tout début.

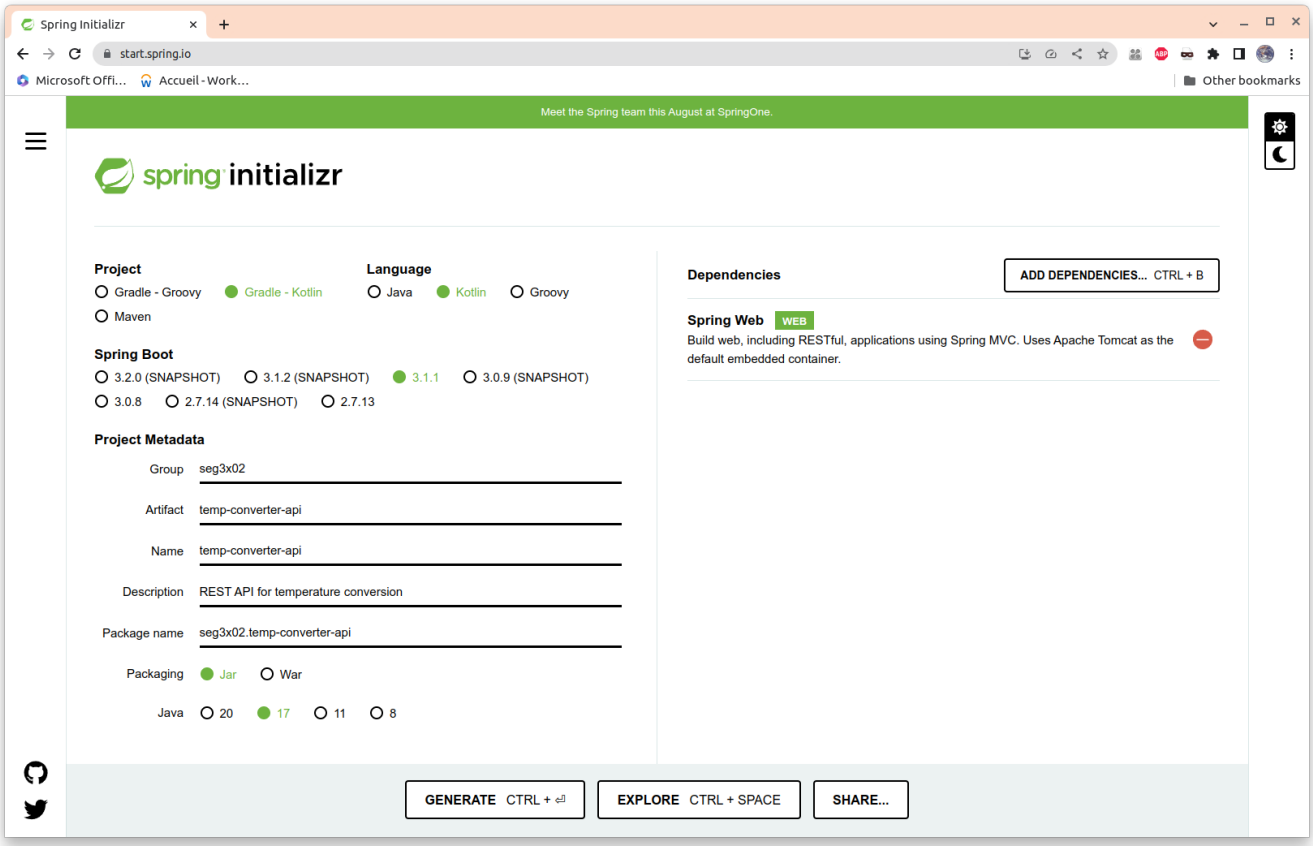
Dans la section suivante, nous présentons une implémentation basée sur le framework Reactive Web, WebFlux introduit depuis la version 5.0 de Spring.

Le code source de l'implémentation Spring MVC du service de conversion de température est disponible à https://github.com/stephanesome/converter-api.git.

Setup du Projet

Créer un projet avec ***Spring Intializr***.

- Sélectionnez *Gradle Project* comme Project type et *Kotlin* comme Language
- Laissez la version Spring Boot sélectionnée par défaut
- Entrez les métadonnées du projet (Group: *seg3x02*, Artifact: *temp-converter-api*)
- Ajoutez la dépendance *Spring Web*.



Générez, téléchargez, décompressez et ouvrez le projet généré dans votre IDE.

OpenAPI

Nous ajoutons une dépendance à la bibliothèque springdoc-openapi, pour générer automatiquement de la documentation de la spécification OpenAPI 3 pour l'API.

Éditez le fichier build.gradle.kts (dans le dossier racine du projet) et ajouter une dépendance org.springdoc:springdoc-openapi-ui. La déclaration des dépendances doit ressembler à ceci:

```
1.  dependencies {
2.      implementation("org.springframework.boot:spring-boot-starter-web")
3.      implementation("org.springdoc:springdoc-openapi-ui:2.8.9")
4.      implementation("com.fasterxml.jackson.module:jackson-module-kotlin")
5.      implementation("org.jetbrains.kotlin:kotlin-reflect")
6.      implementation("org.jetbrains.kotlin:kotlin-stdlib-jdk8")
7.      ...
8.  }
```

La dépendance ajoutée est en ligne 3. Elle configurera la génération automatisée de la documentation OpenAPI basée sur le code.

Contrôleur REST

Créez une classe Kotlin ConverterController dans le package seg3x02.tempconverterapi.controller et modifiez-la comme suit.

```
1.  package seg3x02.tempconverterapi.controller
2.
3.  import org.springframework.web.bind.annotation.GetMapping
4.  import org.springframework.web.bind.annotation.PathVariable
5.  import org.springframework.web.bind.annotation.RequestMapping
6.  import org.springframework.web.bind.annotation.RestController
7.
8.  @RestController
9.  @RequestMapping("temperature-converter")
10. class ConverterController {
11.     @GetMapping("/celsius-fahrenheit/{celsius}")
12.     fun getFahrenheit(@PathVariable celsius: Double) = ((celsius * 9) / 5 + 32)
13.
14.     @GetMapping("/fahrenheit-celsius/{fahrenheit}")
15.     fun getCelsius(@PathVariable fahrenheit: Double) = ((fahrenheit - 32) * 5) / 9
16. }
```

La classe est annotée avec @RestController pour l'enregistrer comme contrôleur pour les requêtes REST. L'annotation @RequestMapping au niveau de la classe spécifie *temperature-converter* comme URI racine pour tous les points de terminaison de service de la classe. Nous définissons deux points de terminaison correspondant aux opérations de service. Chacun est défini comme une fonction annotée avec @GetMapping, une annotation de mappage de requêtes pour la méthode HTTP GET.

Advice de Contrôleur REST

Créez une classe Kotlin ControllerExceptionHandler dans le package seg3x02.tempconverterapi.controller et modifiez-la comme suit.

```
1.  package seg3x02.tempconverterapi.controller
2.
3.  import org.springframework.http.HttpStatus
4.  import org.springframework.http.ResponseEntity
5.  import org.springframework.web.bind.annotation.ExceptionHandler
6.  import org.springframework.web.bind.annotation.ResponseStatus
7.  import org.springframework.web.bind.annotation.RestControllerAdvice
8.
9.  @RestControllerAdvice
10. class ControllerExceptionHandler {
11.     @ExceptionHandler
12.     @ResponseStatus(HttpStatus.BAD_REQUEST)
13.     fun handleException(ex: Exception): ResponseEntity<String> {
14.         return ResponseEntity.badRequest().body("Unable to process request")
15.     }
16. }
```

L’annotation @RestControllerAdvice enregistre la classe comme Controller Advice pour les contrôleurs REST. Nous définissons une fonction de gestion des exceptions (qui gère toute exception ici). La fonction retourne une entité de réponse avec le code d'état BAD_REQUEST (400) et un message d'erreur dans le corps.

Construction et Exécution

Exécutez la commande `./gradlew bootRun` à partir du dossier principal du projet pour générer et exécuter le projet. Vous pouvez également utiliser la commande run de votre IDE.

Une fois l'application démarrée (notez que pour une raison quelconque, l'indicateur de progression avec ./gradlew bootRun reste autour de 80%), vous pouvez vérifier l'API de différentes manières:

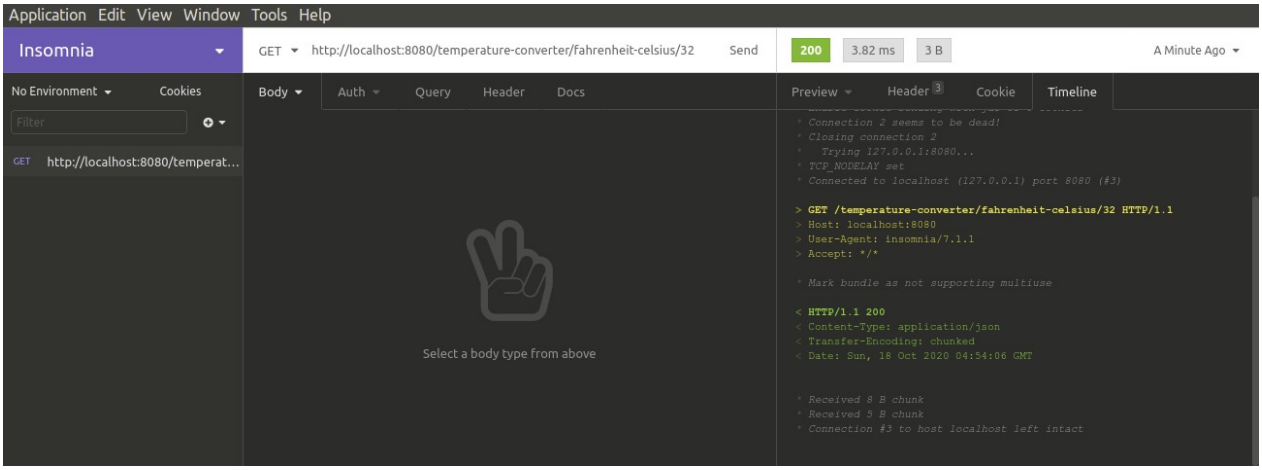
- Avec un navigateur internet, en naviguant à des URL tels que

  http://localhost:8080/temperature-converter/celsius-fahrenheit/100 pour convertir 100 Celsius en Fahrenheit ou http://localhost:8080/temperature-converter/fahrenheit-celsius/-20 pour convertir -20 Fahrenheit en Celsius.

- En utilisant l’outil de ligne de commande curl (https://curl.haxx.se/). Par exemple

  `curl http://localhost:8080/temperature-converter/fahrenheit-celsius/32`

- À l'aide d'un outil graphique tel que Insomnia or Postman. Insomnia est illustrée ci-dessous.

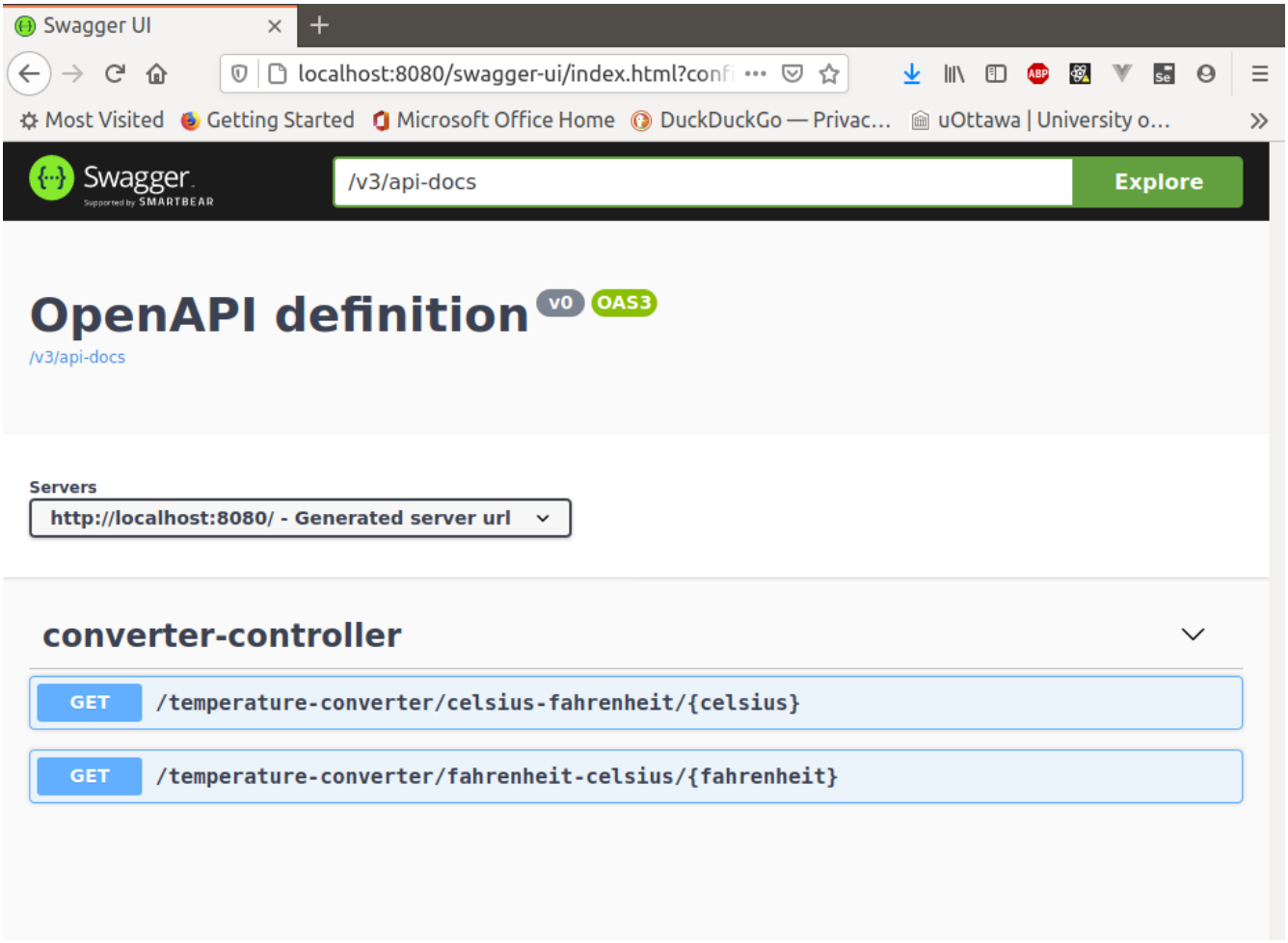


Documentation OpenAPI

Après une exécution de l'application, nous pouvons accéder à la documentation générée par springdoc-openapi dans le format JSON à l'adresse URL http://localhost:8080/v3/api-docs/ .

La documentation dans le format YAML est générée à l'adresse URL `http://localhost:8080/v3/api-docs.yaml`.

Nous pouvons également accéder à la documentation avec Swagger UI à `http://localhost:8080/swagger-ui.html`.



4. Implémentation WebFlux

WebFlux a été introduit dans la version 5.0 de Spring. Contrairement à Spring MVC qui est basé sur les Servlets, WebFlux supporte la programmation asynchrone non bloquante et est destiné à améliorer l'évolutivité et la performance. Il est particulièrement approprié pour le streaming de données et pour supporter des charges élevées. WebFlux est basé sur la spécification Reactive Streams qui s'appuie sur le patron Publisher-Subscriber. Il supporte un modèle de programmation basé sur des annotation ainsi qu'un modèle de programmation fonctionnel/lambda. Nous utilisons le modèle de programmation basé sur les annotations dans cet exemple.

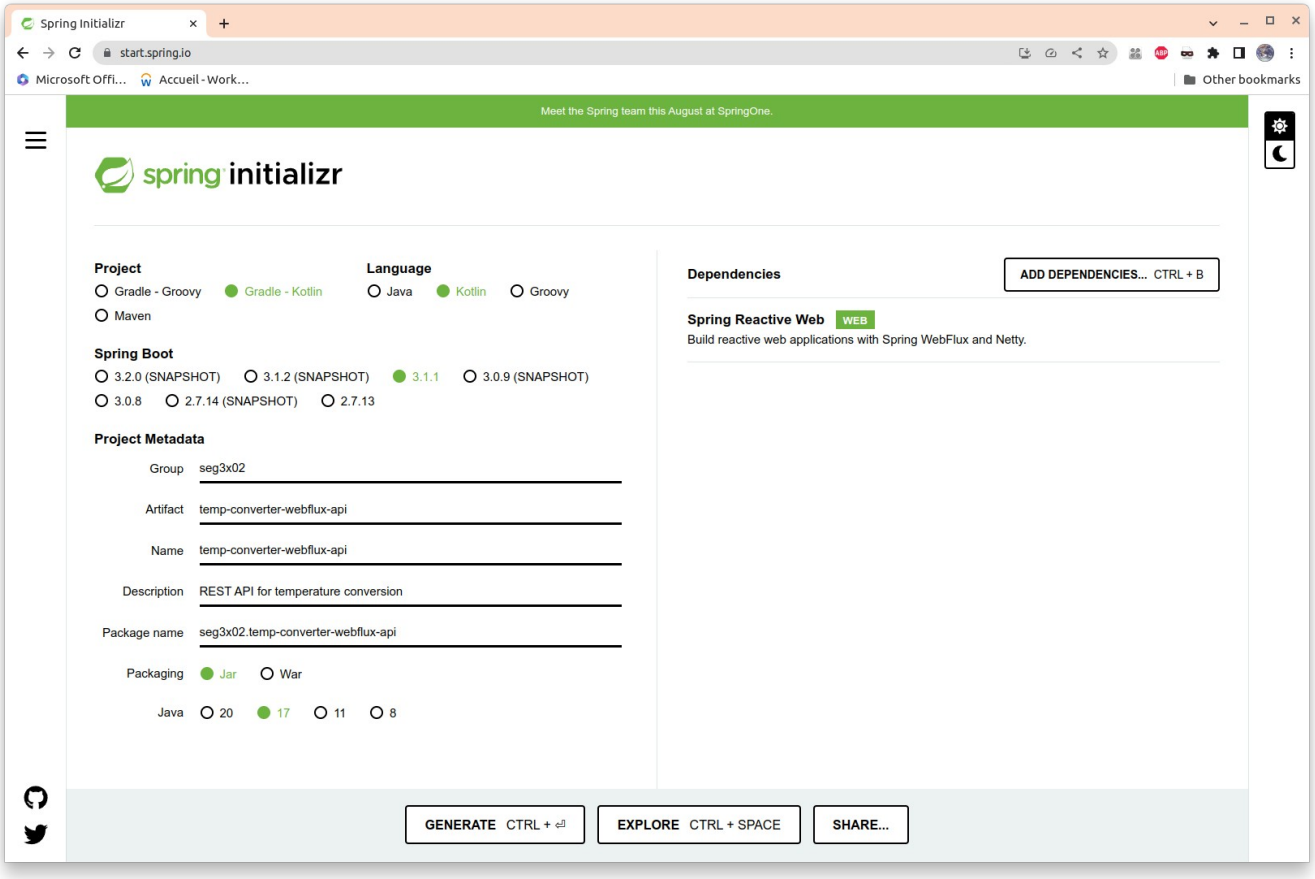
La programmation utilise deux types réactifs composables : Mono et Flux. Un objet Mono représente une valeur unique ou une valeur vide. Un objet Flux représente une séquence réactive de valeurs 0..N. Les objets Mono et Flux sont des publishers auxquels des consommateurs peuvent s'abonner.

Le code source de l'implémentation WebFlux du service de conversion de température est disponible à https://github.com/stephanesome/temp-converter-webflux-api.git.

Configuration du projet

Créez un projet avec ***Spring Intializr***.

- Sélectionnez Gradle Project comme type de projet et Kotlin comme langage
- Laissez la version Spring Boot sélectionnée par défaut
- Entrez les métadonnées du projet (groupe : seg3x02, artefact : temp-converter-webflux-api)
- Ajoutez la dépendance *Spring Reactive Web*.



Notez l'utilisation de la dépendance Spring Reactive Web (spring-boot-starter-webflux).

Générez, téléchargez, décompressez et ouvrez le projet généré dans votre IDE.

`Contrôlleur Réactif`

Le modèle de programmation WebFlux basé sur des annotations utilise les mêmes annotations que le Spring MVC traditionnel.

```
1. package seg3x02.tempconverterwebfluxapi.controller
2.
3. import org.springframework.web.bind.annotation.GetMapping
4. import org.springframework.web.bind.annotation.PathVariable
5. import org.springframework.web.bind.annotation.RequestMapping
6. import org.springframework.web.bind.annotation.RestController
```

```
import reactor.core.publisher.Mono

@RestController
@RequestMapping("temperature-converter")
class ConverterController {
    @GetMapping("/celsius-fahrenheit/{celsius}")
    fun getFahrenheit(@PathVariable celsius: Double) =
                        Mono.just(((celsius * 9) / 5 + 32))

    @GetMapping("/fahrenheit-celsius/{fahrenheit}")
    fun getCelsius(@PathVariable fahrenheit: Double) =
                        Mono.just(((fahrenheit - 32) * 5) / 9)
}
```

```
package seg3x02.tempconverterwebfluxapi.controller

import org.springframework.http.HttpStatus
import org.springframework.http.ResponseEntity
import org.springframework.web.bind.annotation.ExceptionHandler
import org.springframework.web.bind.annotation.ResponseStatus
import org.springframework.web.bind.annotation.RestControllerAdvice
import reactor.core.publisher.Mono

@RestControllerAdvice
class ControllerExceptionHandler {
    @ExceptionHandler
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    fun handleException(ex: Exception): Mono<ResponseEntity<String>> {
        return Mono.just(ResponseEntity.badRequest().
                        body("Unable to process request"))
    }
}
```

La seule différence entre l'implémentation basée sur MVC et l'implémentation réactive est que nous enveloppons les valeurs retournées dans un Mono.

5. Client de l'API de convertisseur

Nous implémentons un client Angular pour l'API RESTful Temperature Converter. Cette implémentation est basée sur l'application Temperature Converter du laboratoire 1. Le code source est disponible à l'adresse https://github.com/stephanesome/temp-converter-api-client.

Setup du Projet

Créez l'application avec la commande `ng new temperature-converter-client`.

Service d'accés à l'API

Générez un service `ng generate service service/converter-service`.

Editez `src/app/service/converter-service.ts` comme suit.

```
1.  import {inject, Injectable} from '@angular/core';
2.  import {HttpClient, HttpErrorResponse} from '@angular/common/http';
3.  import {catchError, Observable, throwError} from 'rxjs';
4.
5.  const baseUrl = 'http://localhost:8080/temperature-converter';
6.
7.  @Injectable({
8.    providedIn: 'root'
9.  })
10. export class ConverterService {
11.   private httpClient: HttpClient = inject(HttpClient);
12.
13.   public convertCelsius(value: string): Observable<number | any> {
14.     return this.httpClient.get(`${baseUrl}/celsius-fahrenheit/${value}`).pipe(
15.       catchError(this.handleError))
16.   }
17.
18.   public convertFahrenheit(value: string): Observable<number | any> {
19.     return this.httpClient.get(`${baseUrl}/fahrenheit-celsius/${value}`).pipe(
20.       catchError(this.handleError))
21.   }
22.
23.   private handleError(error: HttpErrorResponse): Observable<never> {
24.     return throwError(() =>
25.       `Backend returned code ${error.status}, body was: ${error.error}`);
26.   }
27. }
```

Nous spécifions l'URL de base du service Web de conversion de température à la ligne 5.

La fonction convertCelsius (lignes 13-16) convertit les degrés Celsius en degrés Fahrenheit, tandis que la fonction convertFahrenheit (lignes 18-21) convertit les degrés Fahrenheit en degrés Celsius. Toutes deux renvoient un Observable avec le résultat ou l'erreur de conversion. La classe HTTPClient (injectée à la ligne 11) permet d'émettre des requêtes de conversion de température en transmettant l'URL de base ainsi qu'un segment d'URL indiquant l'opération demandée (celsius-fahrenheit ou fahrenheit-celsius) et la valeur à convertir.

Composant TempConverter

Le composant TempConverter est similaire à celui du laboratoire 1, à l'exception de la classe de composant qui utilise désormais le service d'accès à l'API Convertisseur de température.

```
1.  import {Component, inject} from '@angular/core';
2.  import { DecimalPipe } from '@angular/common';
3.  import {ConverterService} from '../service/converter-service';
4.
5.  @Component({
6.    selector: 'app-temp-converter',
7.    templateUrl: './temp-converter.html',
8.    styleUrls: ['./temp-converter.css'],
9.    standalone: true,
10.   imports: [DecimalPipe]
11. })
12. export class TempConverter {
13.   private readonly converterService = inject(ConverterService);
14.   celsiusValue = 0;
15.   fahrenheitValue = 0;
16.
17.   convertCelsius(value: string): void {
18.     this.converterService.convertCelsius(value).subscribe({
19.       next: (response) => { this.fahrenheitValue = response; },
20.       error: (_error) => window.alert(`Error: Unable to convert ${value}`)
21.     });
22.   }
23.
24.   convertFahrenheit(value: string): void {
25.     this.converterService.convertFahrenheit(value).subscribe({
26.       next: (response) => { this.celsiusValue = response; },
27.       error: (_error) => window.alert(`Error: Unable to convert ${value}`)
28.     });
29.   }
30. }
```

Le service est injecté à la ligne 13. Les méthodes convertCelsius (lignes 17-22) et convertFahrenheit (lignes 24-29) invoquent le service d'accès et s'abonnent à l'Observable retourné pour le résultat.

Partage de ressources inter-origines (Cross-Origin Resource Sharing)

Dans cette architecture, le client et le serveur fonctionnent dans des domaines différents. Le partage de ressources inter-origines (CORS) doit être configuré sur le serveur pour permettre la communication inter-domaines.

Ajoutez l'annotation @CrossOrigin à la classe Controller du serveur.

```
1.  @RestController
2.  @CrossOrigin("http://localhost:4200")
3.  @RequestMapping("temperature-converter")
4.  class ConverterController {
5.      ....
6.  }
```

L'annotation indique que le client exécuté à l'adresse http://localhost:4200 est autorisé à envoyer des requêtes inter-domaines au serveur. Dans le cas contraire, ces requêtes seraient refusées.

6. Exercise

L'exercice suivant est le livrable pour le laboratoire. Complétez et enregistrez le code dans Github Classroom avant la date limite. Seul le travail de ce exercice sera évalué.

Développez un service Web Springboot REST Calculator. Le service doit fournir les opérations suivantes:

- GET calculator/add/{nombre1}/{nombre2} pour l'addition
- GET calculator/subtract/{nombre1}/{nombre2} pour la soustraction
- GET calculator/multiply/{nombre1}/{nombre2} pour la multiplication
- GET calculator/divide/{nombre1/{nombre2} pour la division

Ajoutez le nécessaire pour générer la documentation OpenAPI.